

Dalitz Plot Interactive Web Visualizer - Detailed Project Plan

Objective

Develop an interactive web platform for building, visualizing and exploring amplitude models for three-body decays. The application should support both teaching and research.

Reactive workflow

Every user action immediately propagates through the application. Selecting particles, adding resonances, changing masses, widths, lineshapes, magnitudes or phases automatically recalculates the amplitude, Dalitz plot, projections and derived observables. No 'Check' or 'Update' buttons.

Decay selection

The user selects the mother and three daughter particles. The backend queries the particles library to obtain masses and quantum numbers. `qrules` validates the decay, computes the allowed decay topology and the physical Dalitz boundaries. Validation messages are displayed automatically.

Automatic resonance discovery

Once the decay is valid, `qrules` generates the allowed intermediate states. Resonances are grouped by daughter pair and displayed with default parameters from particles.

Custom resonances

Besides suggested resonances, users may add any state manually. They can search the particle database or define a completely custom state with editable name, pair, spin, mass, width, lineshape and complex coefficient. Unsupported states trigger warnings but are never blocked.

Amplitude engine

Each amplitude term is modular: lineshape, spin factor, Blatt-Weisskopf factors and complex coefficient. Initial lineshapes: Relativistic Breit-Wigner, Gounaris-Sakurai and Flatte. Future extensions: K-matrix, LASS and user-defined models.

Visualization

Interactive Plotly figures include Dalitz plot, the three invariant-mass projections, optional individual resonance contributions, interference maps, phase maps, Argand diagrams and fit fractions.

Monte Carlo

Weighted event generation with configurable sample size (up to 1 million events). CSV export contains four-momenta, invariant masses and event weights. Future exports: ROOT and Parquet.

Architecture

Backend: FastAPI. Frontend: React + TypeScript. Physics modules isolated from the user interface. Frontend only edits configuration objects received from the backend.

Design philosophy

Suggest, don't restrict. Everything is editable. Everything is reactive. Physics and presentation remain completely separated.